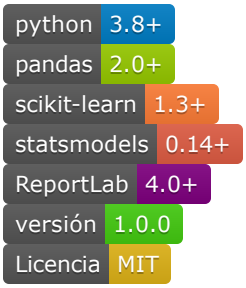
Modelo de Pronóstico de Ventas y Análisis Comercial



> **Proyecto de analítica predictiva y análisis comercial desarrollado en Python, enfocado en la estimación de ventas futuras y en la evaluación ejecutiva del desempeño por producto.**

Tabla de Contenidos

- Descripción General
- Objetivo del Proyecto
- Tecnologías Utilizadas
- Funcionalidades Principales
- Análisis Comercial por Producto
- Arquitectura del Sistema
- Estructura del Proyecto
- Instalación y Configuración
- Ejecución del Proyecto
- Modelos de Pronóstico
- Métricas de Evaluación
- Reportes Generados
- Ejemplo de Resultados
- Casos de Uso
- Mejoras Futuras
- Valor para Portafolio
- Autor y Licencia

Descripción General

Este proyecto integra **dos líneas principales de análisis** que transforman datos de ventas en información estratégica para la toma de decisiones:

1. Pronóstico de Ventas

Permite proyectar ventas futuras utilizando modelos baseline de series temporales, evaluando su desempeño con métricas estándar y generando conclusiones comprensibles para áreas administrativas o comerciales.

```
A[Datos Históricos] --> B[Modelos Baseline]
B --> C[Pronóstico Futuro]
C --> D[Métricas MAE/RMSE/MAPE]
D --> E[Reporte Ejecutivo]
```

2. Análisis Comercial por Producto

Evalúa el comportamiento individual de productos para identificar:

Análisis	Descripción	Beneficio
Mayores ingresos	Productos con más valor monetario	Enfocar inversión
Mayor volumen	Productos con más unidades	Optimizar inventario
Mejor día de venta	Patrón semanal por producto	Campañas específicas
Impacto de promociones	Efectividad de descuentos	Mejorar ROI
Clasificación ABC	Segmentación por contribución	Priorizar gestión
Concentración de ingresos	Dependencia del negocio	Diversificar riesgo
Productos estrella	Alto ingreso + alta rotación	Potenciar
Productos de apoyo	Bajo ingreso + baja rotación	Evaluar continuidad

---

Objetivo del Proyecto

Desarrollar un **flujo reproducible** que transforme datos históricos o simulados en información útil para:

Área de Decisión	Aplicación
Inventario	Optimizar niveles de stock por producto
Inversión comercial	Dirigir recursos a productos clave
Productos críticos	Priorizar gestión de productos A
Temporadas	Anticipar picos y valles de demanda
Incertidumbre	Reducir riesgo en planificación

Tecnologías Utilizadas

PYTHON | PANDAS | NUMPY | SCIKIT-LEARN | STATSMODELS | MATPLOTLIB | REPORTLAB | GITHUB

Tecnología	Versión	Uso Principal
Python	≥3.8	Lenguaje principal
Pandas	≥2.0	Manipulación y análisis de datos
NumPy	≥1.24	Operaciones numéricas
Scikit-learn	≥1.3	Métricas y preprocessing
Statsmodels	≥0.14	Modelos ETS, descomposición
Matplotlib	≥3.7	Visualización de resultados
ReportLab	≥4.0	Generación de PDFs ejecutivos

Funcionalidades Principales

Generación de Datos Ficticios

El sistema puede generar un conjunto de datos simulado con comportamiento comercial realista:

```
# Características de los datos generados
- Tendencia controlada (+2% mensual)
- Variación semanal (fines de semana +30%)
- Estacionalidad (Diciembre +40%)
- Promociones (Black Friday, Cyber Monday)
- Precios por producto (5-500 USD)
- Unidades vendidas (distribución realista)
- 2+ años de datos diarios
- 20+ productos diferentes
```

Preprocesamiento

```
# Pipeline de preparación de datos
1. Validación de columnas requeridas
2. Limpieza de datos (nulos, duplicados)
3. Normalización de formatos
4. Creación de variables temporales
5. Preparación para modelado
```

Modelado de Pronóstico

Se implementan múltiples modelos baseline:

Modelo	Descripción	Uso
Naive	Valor anterior como pronóstico	Baseline mínimo
Promedio Móvil	Media de últimos N períodos	Suavizado de ruido
ETS	Suavizado exponencial	Captura patrones
Regresión Temporal	Variables temporales	Interpretabilidad

Evaluación de Modelos

```
# Métricas implementadas
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

Visualización

Gráficos generados automáticamente:

- Serie histórica completa
- División train/test
- Pronóstico vs real por modelo
- Comparación de desempeño (RMSE)
- Descomposición de serie temporal

Análisis Comercial por Producto

1. Productos con Mayores Ingresos

Identifica los productos que más valor monetario aportan:

```
top_ingresos = df.groupby('producto')
['ingreso'].sum().sort_values(ascending=False).head(10)
```

2. Productos con Mayor Volumen

Detecta productos que mueven más unidades:

```
top_volumen = df.groupby('producto')
['unidades'].sum().sort_values(ascending=False).head(10)
```

3. Mejor Día de Venta por Producto

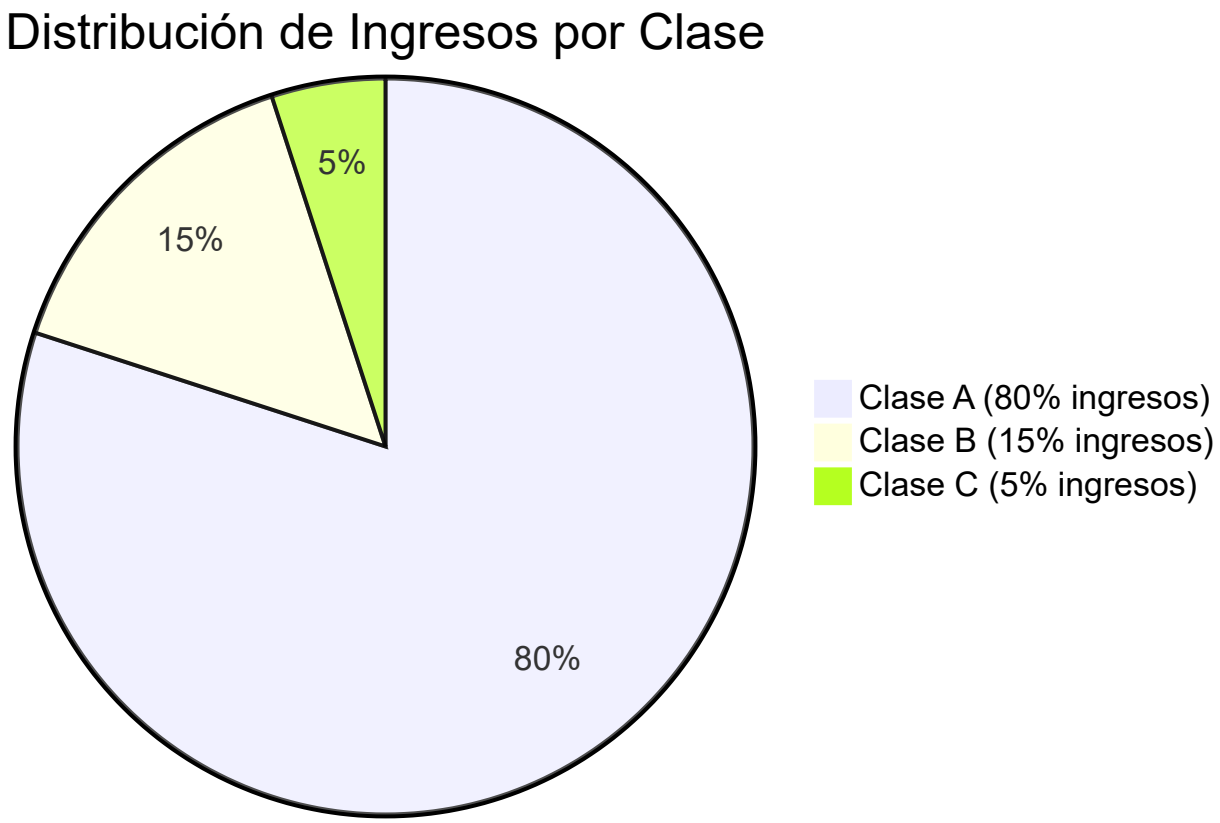
```
# Ejemplo de resultado
Producto A: Sábado (+45% vs promedio)
Producto B: Viernes (+32% vs promedio)
Producto C: Lunes (+15% vs promedio)
```

4. Impacto de Promociones

Producto	Ingreso Promedio (sin promo)	Ingreso Promedio (con promo)	Incremento
Producto A	$1,245	$2,890	+132%

Producto	Ingreso Promedio (sin promo)	Ingreso Promedio (con promo)	Incremento
Producto B	$890	$1,560	+75%
Producto C	$2,100	$3,150	+50%

5. Clasificación ABC



Clase	Criterio	Acción Recomendada
A	80% ingresos acumulados	Gestión prioritaria, stock garantizado
B	15% ingresos acumulados	Gestión regular, monitoreo
C	5% ingresos acumulados	Evaluar continuidad

6. Concentración de Ingresos

```
Índice de Concentración (Top 3 productos): 62%
Índice de Concentración (Top 5 productos): 78%
```

```
Índice de Concentración (Top 10 productos): 92%

Interpretación: El negocio depende fuertemente de pocos productos.
Riesgo: Vulnerabilidad ante problemas con productos clave.
Recomendación: Diversificar portafolio.
```

7. Productos Estrella vs Productos de Apoyo

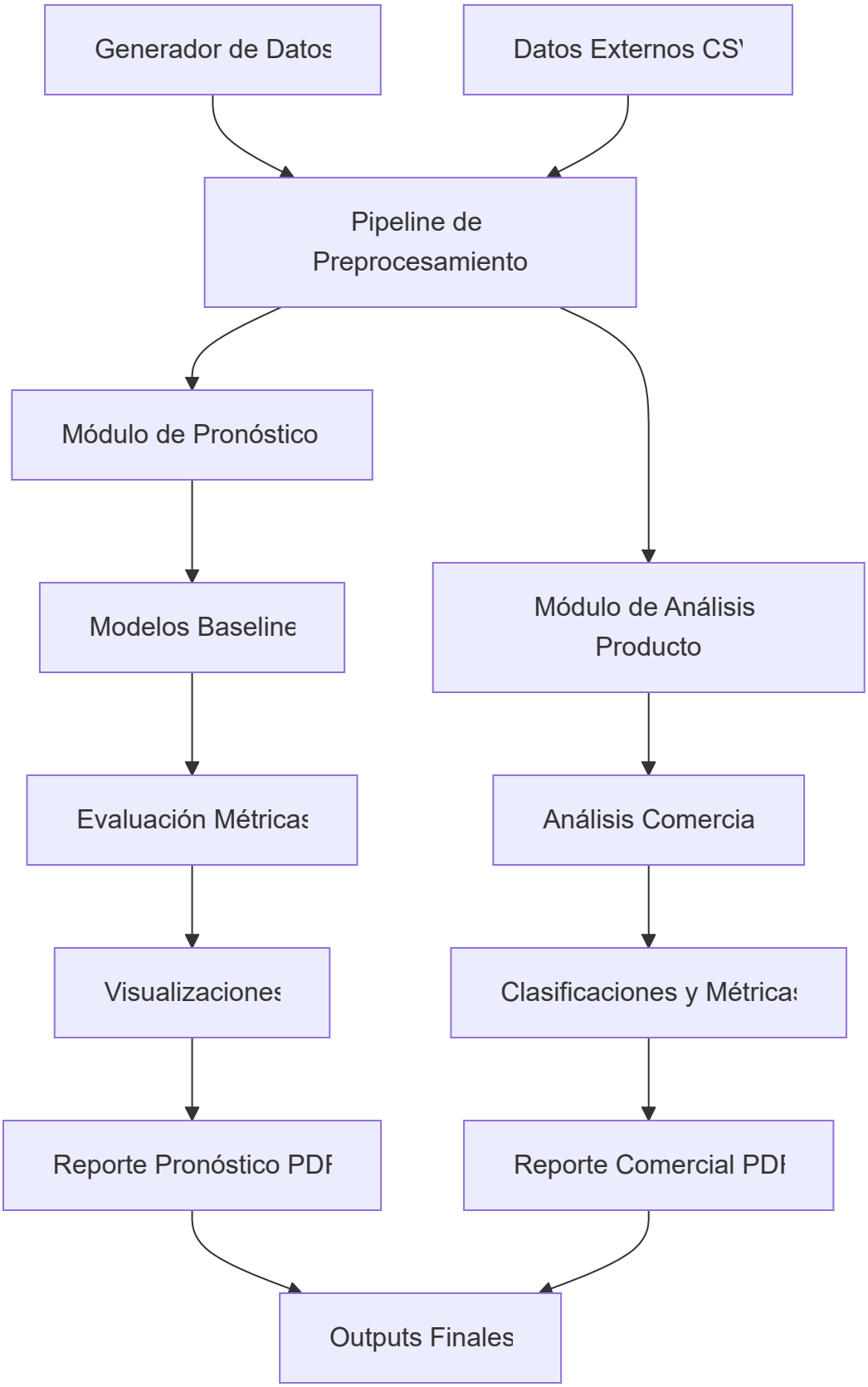
Cuadrante	Ingreso	Rotación	Ejemplo	Acción
Estrella	Alto	Alta	Producto A	Potenciar
Alto Valor	Alto	Baja	Producto B	Optimizar rotación
Apoyo	Bajo	Alta	Producto C	Mantener
Bajo Aporte	Bajo	Baja	Producto D	Evaluar

8. Recomendación de Inventario por Temporada

```
# Recomendaciones generadas automáticamente
TEMPORADA ALTA (Nov-Dic):
  - Aumentar stock 40% para productos A
  - Preparar promociones para productos B
  - Priorizar: Producto A, Producto B, Producto D

TEMPORADA BAJA (Feb-Mar):
  - Reducir inventario 25%
  - Promociones para mantener flujo
  - Enfocar en: Producto C, Producto E
```

Arquitectura del Sistema



Instalación y Configuración

Prerrequisitos

- Python 3.8 o superior
- pip (gestor de paquetes)
- Git (opcional)

Pasos de Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/KPforusesmain/modelo-pronostico-ventas.git
cd modelo-pronostico-ventas

# 2. Crear entorno virtual
python -m venv .venv

# 3. Activar entorno virtual
# En Windows:
.venv\Scripts\activate
# En Linux/Mac:
source .venv/bin/activate

# 4. Instalar dependencias
pip install -r requirements.txt
```

Archivo `requirements.txt`

```
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
statsmodels>=0.14.0
scikit-learn>=1.3.0
reportlab>=4.0.0
jupyter>=6.0.0
seaborn>=0.12.0
openpyxl>=3.1.0
```

Ejecución del Proyecto

Opción 1: Pipeline Completo (Recomendado)

```
# Genera datos, ejecuta análisis y genera reportes
python -m src.main --full_pipeline
```

Opción 2: Solo Pronóstico

```
# Ejecuta solo el módulo de pronóstico
python -m src.main --forecast_only
```

Opción 3: Solo Análisis Comercial

```
# Ejecuta solo el análisis por producto
python -m src.main --product_analysis_only
```

Opción 4: Con Datos Propios

```
# Usar archivo CSV existente
python -m src.main --data_path data/raw/mis_ventas.csv
```

Parámetros de Configuración

Parámetro	Descripción	Default
`--full_pipeline`	Ejecutar todo	`False`
`--forecast_only`	Solo pronóstico	`False`
`--product_analysis_only`	Solo análisis productos	`False`
`--data_path`	Ruta a CSV propio	`None`
`--generate_data`	Generar datos sintéticos	`True`
`--periods`	Períodos a generar	`730` (2 años)
`--forecast_horizon`	Meses a pronosticar	`6`

Modelos de Pronóstico

1. Naive Forecast

```python
def naive_forecast(series, horizon):
    """Último valor conocido como pronóstico"""
    last_value = series.iloc[-1]
    return [last_value] * horizon
```

2. Promedio Móvil

```python
def moving_average_forecast(series, horizon, window=7):
    """Promedio de últimos N días"""
    ma = series.tail(window).mean()
    return [ma] * horizon
```

3. ETS (Exponential Smoothing)

```python
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(
    train,
    seasonal_periods=7,
    trend='add',
    seasonal='add'
)
fitted = model.fit()
forecast = fitted.forecast(horizon)
```

4. Regresión Temporal

```python
# Variables: día_semana, mes, tendencia, promoción
features = ['dayofweek', 'month', 'trend', 'promo']
model = LinearRegression()
model.fit(X_train, y_train)
forecast = model.predict(X_test)
```

Métricas de Evaluación

MAE (Mean Absolute Error)

```python
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
```

```
# Ejemplo: MAE = 245.30 → Error promedio de $245
```

RMSE (Root Mean Square Error)

```
from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
# Ejemplo: RMSE = 312.80 → Penaliza errores grandes
```

MAPE (Mean Absolute Percentage Error)

```
def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
# Ejemplo: MAPE = 8.2% → Error porcentual promedio
```

Casos de Uso

Este proyecto puede aplicarse en diversos contextos:

Caso	Aplicación	Beneficio Principal
Retail	Pronóstico por tienda/categoría	Optimización de inventario
Manufactura	Planificación de producción	Reducción de costos
Logística	Demanda de envíos	Mejor asignación de recursos
Finanzas	Proyección de ingresos	Presupuestos precisos
Marketing	Respuesta a campañas	ROI optimizado
PYMEs	Planificación general	Decisiones informadas
E-commerce	Análisis por producto	Estrategia comercial
Ventas	Identificar estrellas	Foco en productos clave

Mejoras Futuras

Versión 1.1 (Corto Plazo)

- [ ] Interfaz web con Streamlit
- [ ] Más modelos (Prophet, XGBoost, LSTM)
- [ ] Intervalos de confianza en pronósticos
- [ ] Exportación a Excel de todos los análisis

Versión 1.2 (Mediano Plazo)

- [ ] Análisis de elasticidad precio
- [ ] Segmentación de clientes
- [ ] Detección automática de outliers
- [ ] Múltiples granularidades temporales

Versión 2.0 (Largo Plazo)

- [ ] API REST para integración
- [ ] Dashboard interactivo en tiempo real
- [ ] Machine Learning automático (AutoML)
- [ ] Integración con fuentes externas (clima, economía)

Valor para Portafolio

Este proyecto demuestra capacidades en:

Área	Habilidades Demostradas
Análisis de Datos	Pandas, limpieza, transformación avanzada
Series Temporales	ETS, ARIMA, descomposición, estacionalidad
Machine Learning	Scikit-learn, métricas, evaluación comparativa
Visualización	Matplotlib, gráficos interpretables
Python	Programación modular, POO, arquitectura limpia
Business Acumen	Insights accionables, clasificación ABC, recomendaciones
Reporting	Generación de PDF profesional con ReportLab
Documentación	README completo, estructura clara, comentarios

Habilidades Específicas Destacadas

Análisis Avanzado:

- Clasificación ABC con curvas de Pareto
- Matriz de productos (ingreso vs rotación)
- Análisis de estacionalidad por producto
- Impacto cuantitativo de promociones

De Negocio:

- Recomendaciones de inventario por temporada
- Identificación de productos estratégicos
- Alertas de concentración de ingresos
- Insights traducidos a acciones concretas

Técnicas:

- Pipeline completo ETL + análisis + reporting
- Generación de datos sintéticos realistas
- Múltiples formatos de salida (CSV, PNG, PDF)
- Arquitectura modular y escalable

Autor y Licencia

Autor

Proyecto personal desarrollado como parte de un portafolio orientado a **analítica predictiva, inteligencia de negocios y ciencia de datos aplicada**.

GITHUB LINKEDIN

Licencia

Este proyecto está bajo la **Licencia MIT**.
Ver el archivo LICENSE para más detalles.

Referencias y Recursos

- Forecasting: Principles and Practice - Libro de referencia
- Statsmodels Documentation

- Scikit-learn User Guide
- ReportLab User Guide
- Python Data Science Handbook